

Lecture 16: Routing — Distance Vector, Bellman-Ford, and BGP Introduction

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. State the Bellman-Ford equation and explain what each term represents
2. Trace the distance-vector algorithm step by step on a small network, showing how routing tables converge through neighbor exchanges
3. Explain the “good news travels fast, bad news travels slow” asymmetry in DV convergence
4. Describe the count-to-infinity problem and how split horizon and poisoned reverse mitigate it
5. Describe RIP as a deployed distance-vector protocol and identify its key limitations
6. Explain why a single routing algorithm cannot scale to the entire internet
7. Define autonomous system (AS) and distinguish intra-AS routing (IGP) from inter-AS routing (BGP)
8. Describe at a high level how BGP uses path vectors and business relationships to route between autonomous systems

1 Recall: Two Families of Routing

Lecture 15 introduced **link-state routing**: every router floods its local link information so that all routers share a complete topology map, then each router independently runs Dijkstra’s algorithm to compute shortest paths.

Today we examine the alternative: **distance-vector routing**. The two families are, in a sense, duals of each other:

The LS/DV Duality

- **Link-state**: *local information distributed globally*—each router shares what it knows about its own links with every router in the network.
- **Distance-vector**: *global information distributed locally*—each router shares what it knows about all destinations with only its immediate neighbors.

Both families model the network as a weighted graph and both compute shortest paths. They differ fundamentally in what information each router needs, how that information spreads, and how failures are handled.

	Link-State	Distance-Vector
What each router knows	Complete topology	Only neighbor distances
How info spreads	Flood local link info to everyone	Share DVs with neighbors only
Computation	Dijkstra on the full graph	Iterative relaxation via neighbors

2 The Bellman-Ford Equation

Define $d_x(y)$ as the cost of the least-cost path from node x to node y . The **Bellman-Ford equation** states:¹

The Bellman-Ford Equation

$$d_x(y) = \min_{v \in \text{neighbors}(x)} \{c(x, v) + d_v(y)\}$$

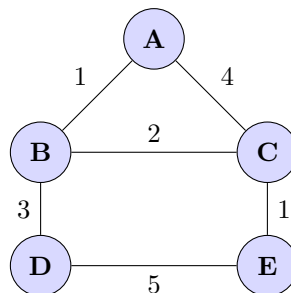
In words: the shortest-path cost from x to y equals the minimum, over all neighbors v of x , of the direct link cost to v plus v 's shortest-path cost to y .

Why It Works: Optimal Substructure

If the shortest path from x to y goes through neighbor v , then the sub-path from v to y must itself be the shortest path from v to y . (If it weren't, we could replace that sub-path with a shorter one and improve the overall path—contradicting optimality.) This is the principle of **optimal substructure**, the foundation of dynamic programming.

Concrete Example

Consider this 5-node network, which we will use throughout the lecture:



¹Wikipedia: Bellman-Ford algorithm—algorithm description, pseudocode, proof of correctness, and relation to dynamic programming.

Example: Bellman-Ford at Node A for Destination E

A's neighbors are B and C. The Bellman-Ford equation gives:

$$d_A(E) = \min \{c(A, B) + d_B(E), c(A, C) + d_C(E)\}$$

We need $d_B(E)$ and $d_C(E)$:

- $d_C(E) = 1$ (direct link)
- $d_B(E) = \min\{c(B, A) + d_A(E), c(B, C) + d_C(E), c(B, D) + d_D(E)\} = \min\{1 + 4, 2 + 1, 3 + 5\} = 3$ (via C)

Therefore:

$$d_A(E) = \min\{1 + 3, 4 + 1\} = 4 \quad \text{via B}$$

Path: $A \rightarrow B \rightarrow C \rightarrow E$, cost $1 + 2 + 1 = 4$. ✓

Bellman-Ford vs. Dijkstra

Dijkstra's algorithm (Lecture 15) requires the **full graph**—all nodes, all edges, all weights—available at one place. Then it runs a centralized algorithm. Bellman-Ford requires only **neighbor information**: each node needs its direct link costs and what its neighbors report as their distances. No node needs the full topology. This is the fundamental advantage of distance-vector routing: it can be computed in a fully **distributed** fashion.

	Dijkstra (L15)	Bellman-Ford (L16)
Information needed	Full graph at one place	Only neighbor info
Computation	Centralized	Distributed
Communication	Flood all links to everyone	Exchange DVs with neighbors
Negative weights	Not supported	Supported
Deployed as	OSPF, IS-IS	RIP

3 The Distance-Vector Algorithm

Each node x maintains a **distance vector**—a table mapping every destination y to x 's current estimate of the shortest-path cost $D_x(y)$, plus the next hop toward y .²

Initialization:

- $D_x(x) = 0$ (cost to yourself is zero)
- $D_x(v) = c(x, v)$ for each neighbor v (direct link cost)
- $D_x(y) = \infty$ for all other nodes y

Algorithm:

1. Each node sends its entire distance vector to all its neighbors.

²Wikipedia: Distance-vector routing protocol—DV overview, Bellman-Ford equation, convergence, comparison with link-state.

2. On receiving D_v from neighbor v : for each destination y , if $c(x, v) + D_v(y) < D_x(y)$, update $D_x(y)$ and set $\text{next_hop}(y) = v$.
3. If any $D_x(y)$ changed, send the updated D_x to all neighbors.
4. Repeat until no distance vector changes (**convergence**).

Properties of the DV Algorithm

- **Iterative**: updates continue until convergence
- **Asynchronous**: nodes do not need to update in lockstep
- **Distributed**: each node communicates only with its direct neighbors
- **Self-terminating**: the algorithm stops when no node has new information to share

Worked Example: Full Convergence Trace

Using the 5-node network from Section 2.

Example: DV Convergence Trace

Round 0 (Initialization) — each node knows only direct links:

	A	B	C	D	E
A	0	1	4	∞	∞
B	1	0	2	3	∞
C	4	2	0	∞	1
D	∞	3	∞	0	5
E	∞	∞	1	5	0

Each node sends its distance vector to its neighbors.

Round 1 — after first exchange (changes in bold):

	A	B	C	D	E
A	0	1	3	4	5
B	1	0	2	3	3
C	3	2	0	5	1
D	4	3	5	0	5
E	5	3	1	5	0

Example update: Node A receives $D_B = [1, 0, 2, 3, \infty]$ from B. $D_A(C) = \min\{4, 1 + 2\} = 3$ via B (improved from 4). $D_A(D) = \min\{\infty, 1 + 3\} = 4$ via B (improved from ∞).

Round 2 — after second exchange:

	A	B	C	D	E
A	0	1	3	4	4
B	1	0	2	3	3
C	3	2	0	5	1
D	4	3	5	0	5
E	4	3	1	5	0

Only two entries improve: $D_A(E)$: $5 \rightarrow 4$ via B (heard $D_B(E) = 3$), and $D_E(A)$: $5 \rightarrow 4$ via C (heard $D_C(A) = 3$).

Round 3: no changes. **Converged.**

Convergence speed: DV converges in at most $|V| - 1$ rounds. The longest shortest path in a graph with $|V|$ nodes has at most $|V| - 1$ edges. Each round extends knowledge by one hop, so after $|V| - 1$ rounds, information has propagated across the entire network. Our 5-node example converged in 2–3 rounds.

4 Good News, Bad News, and Count-to-Infinity

Good News Travels Fast

When a link cost **decreases**, the improvement propagates quickly. Each node that discovers a shorter path immediately tells its neighbors, and the good news cascades through the network. Convergence is fast—proportional to the network diameter.

Bad News Travels Slow

When a link cost **increases** (or a link fails), the problem is much worse. Nodes may continue to believe outdated information from their neighbors, creating a pathological feedback loop.³

Example: Count-to-Infinity (The Two-Node Trap)

Consider a simple chain: $A \xrightarrow{1} B \xrightarrow{1} C$. Initially: $d_A(C) = 2$ via B, $d_B(C) = 1$ direct. The B–C link **fails** (cost $\rightarrow \infty$):

1. B detects the failure. B checks neighbor A: $d_A(C) = 2$. B computes: $D_B(C) = c(B, A) + D_A(C) = 1 + 2 = 3$ via A. **Wrong!** A's route to C goes through B.
2. B tells A: $D_B(C) = 3$. A updates: $D_A(C) = 1 + 3 = 4$. A tells B.
3. B updates: $D_B(C) = 1 + 4 = 5$. B tells A.
4. This continues, incrementing by 1 each round... **counting to infinity**.

During this process, packets for C bounce back and forth between A and B in a **routing loop**.

Root Cause of Count-to-Infinity

Node B uses information from A **that depends on B itself**. A's distance to C goes through B, but B does not know that. There is no mechanism in basic DV to detect this circular dependency.

Fix 1: Split Horizon

Rule: if node A routes to destination C via next-hop B, then A does **not advertise** its route to C back to B.⁴

Intuition: don't tell your neighbor about a route that goes through them. In the example above, A reaches C via B, so A never tells B its $d_A(C) = 2$. When B–C fails, B correctly sets $D_B(C) = \infty$.

Fix 2: Poisoned Reverse

Rule: if node A routes to C via B, then A **actively tells B that** $d_A(C) = \infty$. This is more robust than split horizon—it explicitly kills any temptation for B to route through A.

³Wikipedia: Count-to-infinity problem—formal description, examples, and mitigation techniques.

⁴Wikipedia: Split horizon route advertisement—split horizon and poisoned reverse techniques.

Limitations

Split horizon and poisoned reverse fix the **two-node loop** case. They do **not** prevent count-to-infinity in loops involving **three or more nodes**. Consider a topology where A, B, C, D form a loop: if a link fails, the circular dependency can span multiple intermediate nodes, bypassing split horizon. Larger loops require additional mechanisms such as hold-down timers, route poisoning with triggered updates, or simply accepting that DV convergence can be slow.

	Link-State (OSPF)	Distance-Vector (RIP)
Good news	Fast (one flood + recompute)	Fast (a few rounds)
Bad news	Fast (same mechanism)	Slow (count-to-infinity)
Loop-free?	Yes (full topology)	Not guaranteed during convergence

This asymmetry—fast convergence for good news, slow for bad—is the defining weakness of distance-vector routing and the primary reason OSPF has largely replaced RIP.

5 RIP: Distance-Vector in Practice

The **Routing Information Protocol (RIP)** is the simplest deployed distance-vector protocol.⁵ It implements the algorithm described above with a few practical choices:

Parameter	RIP choice
Metric	Hop count (every link costs 1)
Maximum distance	15 hops ($16 = \infty$)
Update interval	Every 30 seconds, full DV broadcast
Transport	UDP port 520
Triggered updates	Immediate update on route change
Timeout	180 seconds without update $\rightarrow$ all routes via that neighbor set to ∞

Why hop count? It is the simplest possible metric—every link costs 1, so the “shortest path” is the path with the fewest routers. This ignores link capacity, delay, and congestion. A 2-hop path through 10 Gb/s links is treated the same as a 2-hop path through 1 Mb/s links.

Why max 15? Count-to-infinity means that after a failure, costs increment one-by-one until reaching infinity. By defining $\infty = 16$, RIP guarantees convergence in at most 16 rounds (about 8 minutes at 30-second intervals). The tradeoff: RIP cannot be used in networks with paths longer than 15 hops.

RIP’s place today: RIP was the dominant interior routing protocol in the early internet (1980s–1990s). It has been largely replaced by OSPF for most deployments—OSPF converges faster (no count-to-infinity), supports richer metrics (not just hop count), and scales to larger networks (no 15-hop limit). RIP remains in use in small networks and as a teaching protocol.

⁵Wikipedia: Routing Information Protocol—RIP versions 1 and 2, hop count metric, timers, limitations.

Demo: netstat -rn

```
$ netstat -rn
Routing tables

Internet:
Destination      Gateway          Flags    Netif    Metric
default          192.168.1.1     UGS      en0      0
10.0.0.0/24      10.0.0.1        UGS      en1      5
127.0.0.1        127.0.0.1       UH       lo0      0
192.168.1.0/24   link#4          UC       en0      0
```

The **Metric** column is what RIP fills in—the hop count to each destination. OSPF would fill this with computed link-state costs. The routing table structure is protocol-independent.

6 The Scale Problem

Can we run OSPF (or RIP) across the entire internet? The internet has over 75,000 independently administered networks, hundreds of millions of IP prefixes, and diverse policies and business relationships. A single routing algorithm across the whole internet is impossible for three reasons:

1. **Scale:** OSPF requires every router to store the complete topology. With millions of routers and links, the link-state database would be enormous and flooding would generate unsustainable traffic.
2. **Administrative autonomy:** Different organizations need different routing policies. A university, a cloud provider, and a mobile carrier have fundamentally different goals. No single metric satisfies everyone.
3. **Business relationships:** Routing on the internet is driven by economics and contracts, not just shortest paths. An ISP may refuse to carry traffic for a competitor, even if that path is shorter.

The solution is **hierarchical routing**:

- **Within** an organization: run whatever routing algorithm you choose (OSPF, RIP, IS-IS). This is **intra-AS routing**, using an **Interior Gateway Protocol (IGP)**.
- **Between** organizations: run a different protocol designed for policy-based routing across administrative boundaries. This is **inter-AS routing**, using the **Border Gateway Protocol (BGP)**.

7 Autonomous Systems

An **autonomous system (AS)** is a collection of IP networks and routers under the control of a single organization that presents a common routing policy to the internet.⁶ Examples include BU's network, Comcast, Google, and Amazon AWS.

⁶Wikipedia: Autonomous system (Internet)—definition, AS numbers, stub/transit/multi-homed classification.

Each AS is identified by a unique **AS number (ASN)**: originally 16-bit (0–65,535), now extended to 32-bit (up to ~4 billion). ASNs are assigned by Regional Internet Registries (ARIN for North America).

Type	Description	Example	Carries transit?
Stub	Single upstream connection	Small company	No
Multi-homed	Multiple ISPs (redundancy)	University (e.g., BU)	No
Transit	Carries traffic between ASes	Comcast, AT&T	Yes

	Intra-AS (IGP)	Inter-AS (BGP)
Scope	Within a single AS	Between ASes
Goal	Find shortest/cheapest path	Enforce routing policy
Protocols	OSPF, RIP, IS-IS	BGP
Metric	Link cost, hop count, delay	Business relationships
Who decides	Engineer configures link costs	Business contracts determine policy
Convergence	Seconds (OSPF)	Minutes (BGP)

Demo: Identifying ASes on a Path

```
$ traceroute -a google.com
1  [AS0]      192.168.1.1      1.2 ms    (local)
2  [AS7922]   96.120.56.1      8.4 ms    (Comcast)
3  [AS7922]   68.86.103.69     12.1 ms   (still Comcast)
4  [AS15169]  108.170.248.1    14.3 ms   (Google)
5  [AS15169]  142.250.80.46    13.8 ms   (still Google)

$ whois -h whois.radb.net AS15169
as-name:  GOOGLE
descr:    Google LLC
```

Traffic crossed from AS 7922 (Comcast) to AS 15169 (Google). That handoff was determined by **BGP**—the two ASes have a business agreement about which traffic to exchange and where.

8 BGP: The Glue That Holds the Internet Together

The **Border Gateway Protocol (BGP)** is the inter-AS routing protocol of the internet. Every AS that connects to the internet runs BGP.⁷ BGP's job:

1. **Advertise reachability**: “My AS can reach these IP prefixes.”
2. **Propagate paths**: “To reach prefix 128.197.0.0/16, go through AS-path [7922, 3356, 111].”
3. **Enforce policy**: “I will carry this traffic” or “I refuse.”

⁷Wikipedia: Border Gateway Protocol—BGP overview, path vector, AS-PATH, route selection process.

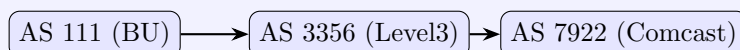
Path Vector: A Third Approach

BGP is neither link-state nor pure distance-vector. It is a **path vector** protocol:

	Distance Vector	Link State	Path Vector (BGP)
Advertises	Distance to dest	Full local link info	Complete AS-path to prefix
Loop prevention	Split horizon	Full topology	Reject own ASN in path
Metric	Numeric cost	Numeric cost	Policy-based

When a BGP speaker receives a route advertisement, the **AS-PATH** attribute lists every AS the advertisement has traversed. If a router sees its own ASN in the path, it **rejects the route**—this is how BGP prevents routing loops, cleanly and without count-to-infinity.

Example: AS-PATH



Prefix: 128.197.0.0/16 (BU's address space). AS-PATH: [111, 3356, 7922]—each AS the route has passed through. If Comcast receives the route back with its own ASN 7922 in the path, it rejects it.

BGP and Business Relationships

Unlike OSPF or RIP, BGP routing decisions are driven by **business relationships**, not shortest paths.⁸ The three fundamental relationships:

- **Customer–Provider:** the customer pays the provider for internet access. The provider advertises the customer's routes to the rest of the internet.
- **Peer–Peer:** two ASes agree to carry each other's traffic for free—but only traffic destined for each other's customers, not transit to the broader internet.

The Valley-Free Rule

Traffic flows **up** from customer to provider, across a **peering link** (at most once), then **down** from provider to customer. A well-configured BGP system never routes customer→provider→peer→provider→customer—that would mean the middle providers are giving away free transit.

Why BGP Is Hard

BGP is arguably the most complex protocol in the internet:

- **Policy over optimality:** the “best” route is not the shortest—it's the one that satisfies business contracts and policy constraints.
- **No global view:** unlike OSPF, no single entity sees the full internet routing table with all policies.

⁸Wikipedia: Peering—peering vs. transit, Internet exchange points, business relationships.

- **Slow convergence:** BGP convergence after a failure can take **minutes** (compared to seconds for OSPF).
- **Scale:** the global BGP routing table contains over **1 million prefixes**.
- **Trust-based:** ASes trust their neighbors' announcements, and there is limited infrastructure to verify them. (Efforts like RPKI and BGPsec aim to add verification.)

BGP Incidents

BGP's power comes with risk.⁹ Notable incidents:

- **Pakistan/YouTube (2008):** Pakistan Telecom announced a more-specific prefix for YouTube's address space. The announcement propagated globally, redirecting YouTube traffic into a black hole for ~ 2 hours. This exploits longest-prefix match (Lecture 14): a $/24$ is preferred over a $/22$.
- **Facebook outage (2021):** A configuration change withdrew BGP routes for all Facebook properties. Because Facebook's DNS servers were also behind those routes, the domain names themselves became unresolvable. The outage lasted ~ 6 hours.

9 The Complete Routing Picture

We now have the full routing landscape for this course:

Lecture	Algorithm	Type	Scope	Deployed as
L15	Dijkstra	Link-state	Intra-AS	OSPF, IS-IS
L16	Bellman-Ford	Distance-vector	Intra-AS	RIP
L16	Path vector	Policy-based	Inter-AS	BGP

Inside an AS, OSPF (or RIP in small networks) computes shortest paths. Between ASes, BGP enforces policy and propagates reachability. A packet crossing the internet uses both: intra-AS routing within each AS it traverses, and inter-AS routing to decide which AS to enter next.

Consider...	Link-State (OSPF)	Distance-Vector (RIP)
Network size	Large (100s of routers)	Small (< 15 hops)
Convergence speed	Fast	Slow (count-to-infinity)
Memory/CPU	Higher (stores full topology)	Lower (stores only DV)
Implementation complexity	More complex	Simpler
Metric flexibility	Rich (bandwidth, delay)	Typically hop count only

Optional further reading on BGP: BGP mechanics (eBGP vs. iBGP, route selection), hot potato routing, RPKI/BGPsec, BGP looking glasses (BGP.he.net), RIPE Stat, RouteViews.

⁹Wikipedia: BGP hijacking—prefix hijacking incidents, mitigation approaches.

Discussion Questions

1. In the distance-vector algorithm, why does each node send its *entire* distance vector to neighbors, rather than just the entries that changed? What are the tradeoffs?
2. Consider a network where all links have cost 1. Compare how Dijkstra and Bellman-Ford compute shortest paths. Which requires more rounds of communication? Which requires more information at each node?
3. In the count-to-infinity example, costs increment by 1 each round. If the original path cost was 100, how many rounds would count-to-infinity take with RIP's max of 16? What if we raised the infinity value to 256?
4. Construct a 4-node example where split horizon fails to prevent count-to-infinity. (Hint: create a loop involving three or more nodes.)
5. BGP uses AS-PATH for loop prevention: a router rejects any route containing its own ASN. Why is this more robust than split horizon?
6. A university (stub AS) is connected to two ISPs. If one ISP link fails, how does the rest of the internet learn to route through the other ISP? How long might this take?
7. Why doesn't the internet use link-state routing globally? What specific technical and organizational barriers prevent running one giant OSPF instance?
8. The Pakistan/YouTube incident happened because a more-specific prefix (/24) was preferred over a less-specific one (/22). Using your knowledge of longest-prefix match from L14, explain why. How could this be prevented?

Further Reading

The following Wikipedia articles are the primary student-facing references for this lecture.

- Wikipedia: Bellman–Ford algorithm—algorithm description, pseudocode, proof of correctness, relation to dynamic programming.
- Wikipedia: Distance-vector routing protocol—DV overview, Bellman-Ford equation, count-to-infinity, comparison with link-state.
- Wikipedia: Routing Information Protocol—RIP versions 1 and 2, hop count metric, timers, limitations.
- Wikipedia: Count-to-infinity problem—formal description, split horizon, poisoned reverse.
- Wikipedia: Split horizon route advertisement—split horizon and poisoned reverse techniques.
- Wikipedia: Autonomous system (Internet)—AS definition, ASNs, stub/transit/multi-homed classification.
- Wikipedia: Border Gateway Protocol—BGP overview, path vector, AS-PATH, route selection.

- Wikipedia: BGP hijacking—prefix hijacking incidents, Pakistan/YouTube 2008, mitigation approaches.
- Wikipedia: Peering—peering vs. transit, IXPs, business relationships between ASes.
- Wikipedia: Interior gateway protocol—IGP vs. EGP classification; OSPF, IS-IS, RIP comparison.
- Wikipedia: Hot-potato routing—hot-potato and cold-potato routing between autonomous systems.

Prepared for EC 441, Boston University, Spring 2026.